

Foundations of Diffusion model

[Diffusion model]

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

1.0 Content Block

"network", because f_θ is implemented as a neural network. "score", because the output of the network is interpreted as approximating the score function $\nabla \ln p(x)$. "noise conditional", because p_t is equal to p_0 blurred by an added Gaussian noise that increases with time, and so the score function depends on the amount of noise added.

2.0 Content Block

$\nabla_x \ln p(x|y) = \nabla_x \ln p(x) \text{ score} + \nabla_x \ln p(y|x) \text{ classifier guidance}$
 $\nabla_x \ln p(x|y) = \underbrace{\nabla_x \ln p(x)}_{\text{score}} + \underbrace{\nabla_x \ln p(y|x)}_{\text{classifier guidance}}$

3.0 Content Block

Forward diffusion process Consider again the forward diffusion process, but this time in continuous time: $x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} z_t$. By taking the $\beta_t \rightarrow \beta(t) dt$, $dz_t \rightarrow dW_t$ limit, we obtain a continuous diffusion process, in the form of a stochastic differential equation: $dx_t = -\frac{1}{2} \beta(t) x_t dt + \sqrt{\beta(t)} dW_t$ where W_t is a Wiener process (multidimensional Brownian motion). Now, the equation is exactly a special case of the overdamped Langevin equation $dx_t = -\frac{D}{k_B T} (\nabla_x U) dt + \sqrt{2D} dW_t$ where D is diffusion tensor, T is temperature, and U is potential energy field. If we substitute in $D = \frac{1}{2} \beta(t) I$, $k_B T = 1$, $U = \frac{1}{2} \|x\|^2$, we recover the above equation. This explains why the phrase "Langevin dynamics" is sometimes used in diffusion models. Now the above equation is for the stochastic motion of a single particle. Suppose we have a cloud of particles distributed according to q at time $t=0$, then after a long time, the cloud of particles would settle into the stable distribution of $\mathcal{N}(0, I)$. Let p_t be the density of the cloud of particles at time t , then we have $p_0 = q$; $p_T \approx \mathcal{N}(0, I)$ and the goal is to somehow reverse the process, so that we can start at the end and diffuse back to the beginning. By Fokker-Planck equation, the density of the cloud evolves according to $\partial_t \ln p_t = \frac{1}{2} \beta(t) (n + (x + \nabla \ln p_t) \cdot \nabla \ln p_t + \Delta \ln p_t)$ where n is the dimension of space, and Δ is the Laplace operator. Equivalently, $\partial_t p_t = \frac{1}{2} \beta(t) (\nabla \cdot (x p_t) + \Delta p_t)$

4.0 Content Block

Noise prediction network Since $x_{t-1} | x_t, x_0 \sim \mathcal{N}(\mu_t(x_t, x_0), \sigma_t^2 I)$, this suggests

that we should use $\mu_{\theta}(x_t, t) = \mu_{\sim t}(x_t, x_0)$ $\{\displaystyle \mu_{\theta}(x_t, t) = \tilde{\mu}_{\sim t}(x_t, x_0)\}$; however, the network does not have access to x_0 $\{\displaystyle x_0\}$, and so it has to estimate it instead. Now, since $x_t | x_0 \sim N(\alpha^{-t} x_0, \sigma_t^2 I)$ $\{\displaystyle x_t | x_0 \sim N(\alpha^{-t} x_0, \sigma_t^2 I)\}$, we may write $x_t = \alpha^{-t} x_0 + \sigma_t z$ $\{\displaystyle x_t = \sqrt{\bar{\alpha}_t} x_0 + \sigma_t z\}$, where z $\{\displaystyle z\}$ is some unknown Gaussian noise. Now we see that estimating x_0 $\{\displaystyle x_0\}$ is equivalent to estimating z $\{\displaystyle z\}$. Therefore, let the network output a noise vector $\epsilon_{\theta}(x_t, t)$ $\{\displaystyle \epsilon_{\theta}(x_t, t)\}$, and let it predict $\mu_{\theta}(x_t, t) = \mu_{\sim t}(x_t, x_t - \sigma_t \epsilon_{\theta}(x_t, t) \alpha^{-t}) = x_t - \epsilon_{\theta}(x_t, t) \beta_t / \sigma_t \alpha_t$ $\{\displaystyle \mu_{\theta}(x_t, t) = \tilde{\mu}_{\sim t}(x_t, \frac{x_t - \sigma_t \epsilon_{\theta}(x_t, t)}{\alpha_t}) = \frac{x_t - \epsilon_{\theta}(x_t, t) \beta_t}{\sigma_t \alpha_t}\}$ $\{\displaystyle \mu_{\theta}(x_t, t) = \tilde{\mu}_{\sim t}(x_t, \frac{x_t - \sigma_t \epsilon_{\theta}(x_t, t)}{\alpha_t}) = \frac{x_t - \epsilon_{\theta}(x_t, t) \beta_t}{\sigma_t \alpha_t}\}$. It remains to design $\Sigma_{\theta}(x_t, t)$ $\{\displaystyle \Sigma_{\theta}(x_t, t)\}$. The DDPM paper suggested not learning it (since it resulted in "unstable training and poorer sample quality"), but fixing it at some value $\Sigma_{\theta}(x_t, t) = \zeta_t^2 I$ $\{\displaystyle \Sigma_{\theta}(x_t, t) = \zeta_t^2 I\}$, where either $\zeta_t^2 = \beta_t$ or $\sigma_{\sim t}^2$ $\{\displaystyle \zeta_t^2 = \beta_t \text{ or } \sigma_{\sim t}^2\}$ yielded similar performance. With this, the loss simplifies to $L_t = \beta_t^2 \alpha_t \sigma_t^2 \zeta_t^2 E_{x_0 \sim q; z \sim N(0, I)} [\|\epsilon_{\theta}(x_t, t) - z\|^2] + C$ $\{\displaystyle L_t = \frac{\beta_t^2}{\alpha_t \sigma_t^2 \zeta_t^2} E_{x_0 \sim q; z \sim \mathcal{N}(0, I)} [\|\epsilon_{\theta}(x_t, t) - z\|^2] + C\}$ which may be minimized by stochastic gradient descent. The paper noted empirically that an even simpler loss function $L_{\text{simple}, t} = E_{x_0 \sim q; z \sim N(0, I)} [\|\epsilon_{\theta}(x_t, t) - z\|^2]$ $\{\displaystyle L_{\text{simple}, t} = E_{x_0 \sim q; z \sim \mathcal{N}(0, I)} [\|\epsilon_{\theta}(x_t, t) - z\|^2]\}$ resulted in better models.